



HARDWARE MODELING

Hardware Modeling Languages • Logic Networks • State Diagrams
Data-Flow Graphs • Sequencing Graphs • Behavioral Optimization
7th Semester CSE

What is Hardware Modeling?

Hardware Modeling describes **how a digital hardware system works**.

A system can be represented using **diagrams, graphs, equations, or HDL code**.

It helps us **understand, simulate, analyze, and improve a design**.

Hardware → Model → Simulate → Optimize → Implement

Why Do We Need Hardware Modeling?

- Makes complex hardware easier to understand.
- Helps identify design errors before implementation.
- Allows simulation and verification.
- Helps improve speed, area, and power consumption.
- Useful before FPGA or ASIC implementation.

Hardware Modeling Languages (HDLs)

Hardware Description Language (HDL) is used to describe digital hardware.

An HDL can describe hardware structure, behavior, and data flow.

Common HDLs: VHDL, Verilog, SystemVerilog.

HDL = Language for Describing Hardware

HDL vs Programming Language

Programming Language

Software তৈরি করে

Sequential execution বেশি গুরুত্বপূর্ণ

C, C++, Java

CPU-তে program চলে

Hardware Description Language

Hardware describe করে

Parallel operation গুরুত্বপূর্ণ

VHDL, Verilog

Circuit hardware হিসেবে implement হতে পারে

Three Ways to Model Hardware

Structural Modeling → components + connections

Data-Flow Modeling → how data moves

Behavioral Modeling → what the system does

Example: AND gate + OR gate + NOT gate

Structural Modeling

Describes the components of a circuit and their connections.

Example: Half Adder uses XOR and AND gates.

$$\text{Sum} = A \oplus B$$

$$\text{Carry} = A \cdot B$$

Structural modeling হলো:

"Circuit-এর ভিতরে কোন কোন component আছে এবং তারা কীভাবে connected?"

Example

Half Adder:

Inputs: A, B

Outputs:

- Sum
- Carry

Logic:

$$\text{Sum} = A \oplus B$$

$$\text{Carry} = A \cdot B$$

এখানে XOR এবং AND gate ব্যবহার করা হয়েছে।

A, B → XOR → Sum | A, B → AND → Carry

Data-Flow Modeling → How Data Moves

Data-Flow Modeling describes how data moves from one operation or component to another inside a digital system.

It mainly focuses on:

Input → Operation → Intermediate Data → Output

Example: $Y = (A + B) \times C$

Suppose we want to calculate:

$$Y = (A + B) \times C$$

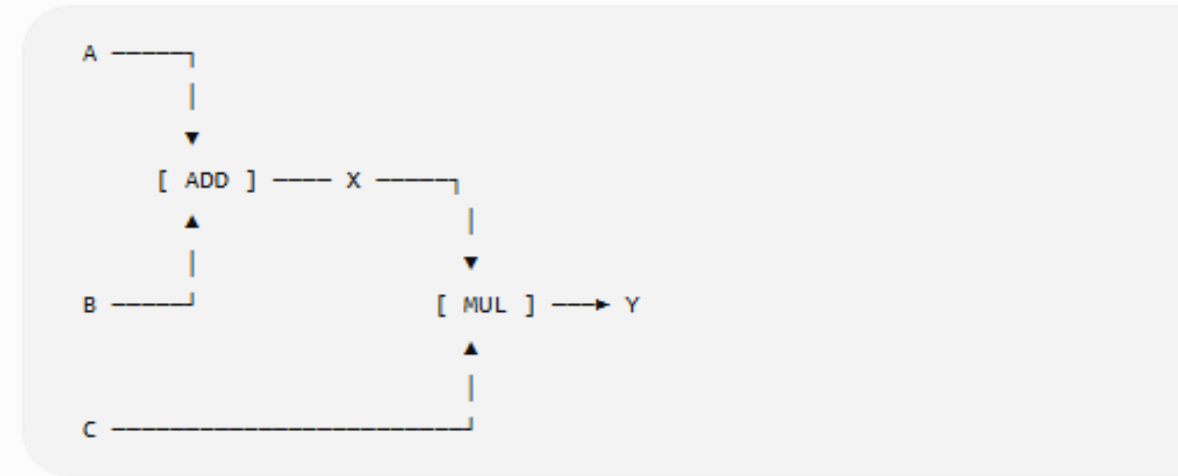
First, we add **A** and **B**:

$$X = A + B$$

Then, we multiply the result by **C**:

$$Y = X \times C$$

Data-Flow Diagram



What does the diagram tell us?

- **A** and **B** are input data.
- ADD operation produces intermediate data **X**.
- **X** and **C** go to the MUL operation.
- MUL produces final output **Y**.

Therefore:

$A, B \rightarrow ADD \rightarrow X \rightarrow MUL \rightarrow Y$

Easy way to remember

Data-Flow Modeling = "Where does the data go?"

Behavioral Modeling → **What the System Does**

Behavioral Modeling describes what a hardware system does for given inputs, without focusing on the detailed internal gate connections.

It mainly focuses on:

Input → Behavior/Operation → Output

Example 1: AND Gate

Instead of drawing an AND gate using transistors/gates, we simply describe its behavior:

$$Y = A \cdot B$$

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

The behavioral description is:

The output Y becomes 1 only when both A and B are 1.

We do not need to describe how the transistors are physically connected.

Behavioral Modeling → What the System Does

Example 2: 2-to-1 Multiplexer

A 2-to-1 MUX has:

- Inputs: A, B
- Select: S
- Output: Y

Its behavior is:

$$Y = \begin{cases} A, & S = 0 \\ B, & S = 1 \end{cases}$$

Truth Table

S	Y
0	A
1	B

Behavioral explanation

If:

$S = 0 \rightarrow \text{Output} = A$

$S = 1 \rightarrow \text{Output} = B$

So we are describing **what the MUX does**, rather than how its internal gates are connected.

Example 3: Adder

Suppose a circuit adds two numbers:

$$Y = A + B$$

Behavioral description:

“The circuit receives A and B and produces their sum at the output Y .”

For example:

$$A = 5, \quad B = 3$$

Therefore:

$$Y = 5 + 3 = 8$$

We don't need to show the individual AND, OR, XOR gates to describe this behavior.

Data-Flow vs Behavioral Modeling

Data-Flow Modeling

Shows **how data moves**

Focuses on operations and data dependencies

Uses data-flow representation

Example: ADD → MUL

Question: **Where does data go?**

Behavioral Modeling

Shows **what the system does**

Focuses on system behavior

Uses equations, algorithms, or HDL statements

Example: $Y = (A + B) \times C$

Question: **What should the system do?**

Very Easy Memory Trick

Data-Flow = HOW DATA MOVES

Behavioral = WHAT SYSTEM DOES

In HDL

For example, behavioral description of a simple AND operation can be written as:

```
</> verilog  
  
assign Y = A & B;
```



Here we are saying **what output should be produced from the inputs**, rather than describing the physical transistor-level implementation.

Logic Networks

A logic network is a **group of connected logic gates**.

Basic gates: AND, OR, NOT, NAND, NOR, XOR, XNOR.

Logic networks implement Boolean functions.

$A, B \rightarrow \text{AND} \rightarrow X \quad | \quad X, C \rightarrow \text{OR} \rightarrow Y$

Logic Network Example

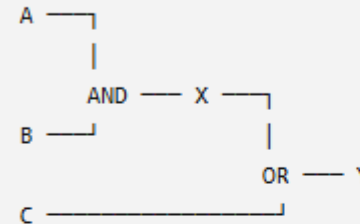
Function: $Y = A \cdot B + C$

Step 1: AND gate produces $X = A \cdot B$

Step 2: OR gate produces $Y = X + C$

Therefore: $Y = A \cdot B + C$

Circuit



Step-by-step

Step 1: A এবং B $\rightarrow$ AND

$$X = A \cdot B$$

Step 2: X এবং C $\rightarrow$ OR

$$Y = X + C$$

Therefore,

$$Y = A \cdot B + C$$

$A, B \rightarrow \text{AND} \rightarrow X \rightarrow \text{OR} \rightarrow Y$ (C also enters OR)

What is a State?

A state represents the current condition of a digital system.

A system may move from one state to another because of an input or event.

Example: Traffic light → RED, GREEN, YELLOW.

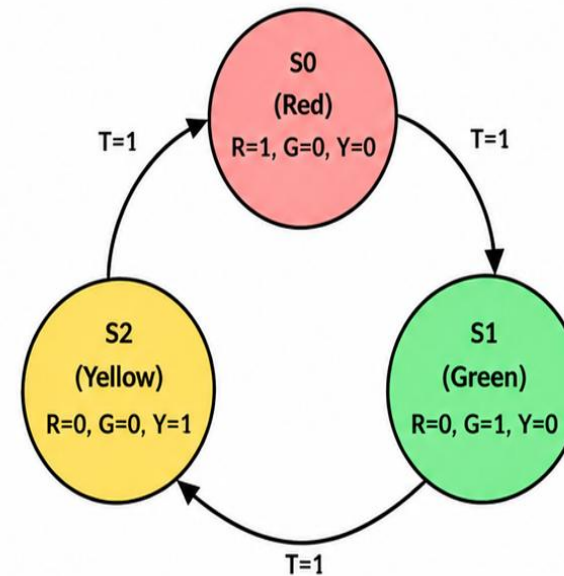
State = Current condition of the system

State Diagram

Definition:

A state diagram is a graphical representation of a sequential system, showing the different **states** of the system and the **transitions** between them based on inputs.

Example: Simple Traffic Light Controller



Legend

- S_i = State (with output)
- $\rightarrow$ = Transition (with input condition)
- R, G, Y = Output signals (Red, Green, Yellow)
- T = Input (Timer / condition)

State Transition Table

Current State	Input (T)	Next State	Output (R, G, Y)
S0 (Red)	1	S1 (Green)	1, 0, 0
S1 (Green)	1	S2 (Yellow)	0, 0, 1
S2 (Yellow)	1	S0 (Red)	0, 1, 0

Explanation

- Each circle represents a **state** of the system.
- The label inside the circle shows the **state name** and its **output**.
- Arrows show the **transition** from one state to another based on the input condition (T).
- In this example, when $T = 1$, the system moves from **Red** → **Green** → **Yellow** → **Red**, continuously.

Key Points

- Used in FSM (Finite State Machine)
- Helps in designing controllers
- Common in traffic lights, vending machines, sequence detectors, and many digital systems.

State Diagram

A state diagram graphically represents states and transitions.

State → circle/rounded box

Arrow → transition

Input condition → written near the transition

S0 —(Input = 1)—> S1 | S1 —(Input = 0)—> S0

State Diagram Example

S0 = No Coin

S1 = Coin Inserted

S0 → S1 when a coin is inserted.

S1 → S0 after the product is delivered.

S0 (No Coin) —Coin—> S1 (Coin Inserted)

Data-Flow Graph (DFG)

A Data-Flow Graph shows how data moves between operations.

Nodes represent operations.

Arrows represent data movement or dependency.

$A, B \rightarrow \text{ADD} \rightarrow X \rightarrow \text{MUL} \leftarrow C \rightarrow Y$

DFG Example

$$Y = (A + B) \times C$$

First operation: $X = A + B$

Second operation: $Y = X \times C$

ADD and MUL are operation nodes; arrows show dependency.

A,B → ADD → X → MUL ← C → Y

ধরি:

$$Y = (A + B) + (C - D)$$

প্রথমে:

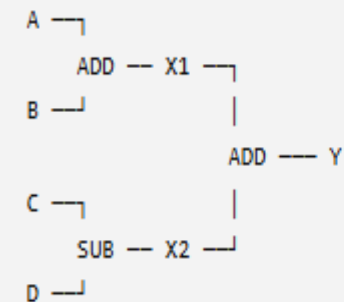
$$X_1 = A + B$$

$$X_2 = C - D$$

তারপর:

$$Y = X_1 + X_2$$

Graph



বিষয়	DFG – Data-Flow Graph	DFD – Data Flow Diagram
মূল ব্যবহার	Hardware/digital system-এর ভিতরে data কীভাবে process হয়	Software/information system-এ data কোথা থেকে কোথায় যায়
Focus	Operations + data dependency	Processes + data movement
সাধারণত	Hardware design, compiler, DSP, VLSI	Software/system analysis
Node	ADD, MUL, SUB ইত্যাদি operation	Process, database, external entity
Example	$(A+B) \times C$	User → Process → Database
Diagram	$A, B \rightarrow \text{ADD} \rightarrow X \rightarrow \text{MUL} \rightarrow Y$	User → Process → Database

Sequencing Graph

A Sequencing Graph shows the order in which operations are performed.

It answers: Which operation happens first, next, and last?

Example:

Read Data → Add → Multiply → Store Result

Data-Flow Graph vs. Sequencing Graph

DFG → focuses on data movement and dependency.

Sequencing Graph → focuses on execution order.

Easy memory: DFG = DATA flow; Sequencing = ORDER.

**DFG → Where does data go? |
Sequencing → When does it happen?**

Sequencing Graph দেখায়:

কোন operation আগে হবে এবং কোন operation পরে হবে।

Example

ধরি:

```
graph TD; A[Read Data] --> B[Add]; B --> C[Multiply]; C --> D[Store Result];
```

এখানে sequence হলো:

Read → Add → Multiply → Store

Behavioral Optimization

Optimization means **improving a hardware design**.

Common goals: **reduce delay, area, power, and unnecessary operations**.

A good design should meet **required performance with efficient hardware**.

Optimization Example

Before: $Y = A \cdot B + A \cdot C$

After: $Y = A(B + C)$

Common factor A is extracted.

Possible benefits: less repeated hardware, lower area/power, simpler implementation.

Question: For the Boolean function

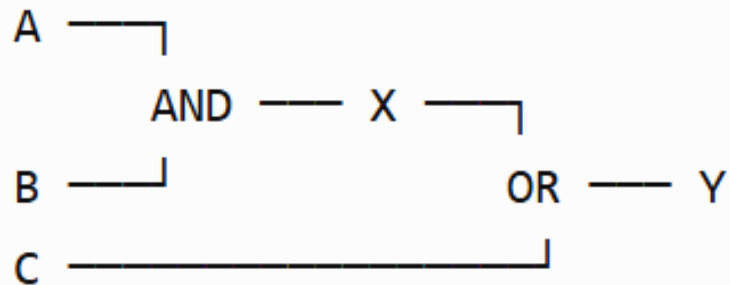
$$Y = (A \cdot B) + C$$

Draw the Logic Network and Data-Flow Graph (DFG).

First, perform $A \cdot B$ using an AND gate.

Then, perform $(A \cdot B) + C$ using an OR gate.

Logic Network:

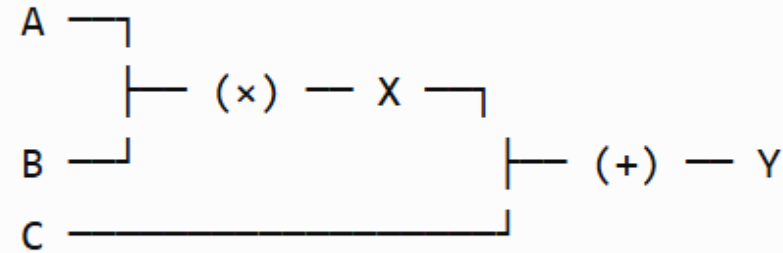


Where,

$$X = A \cdot B$$

$$Y = X + C$$

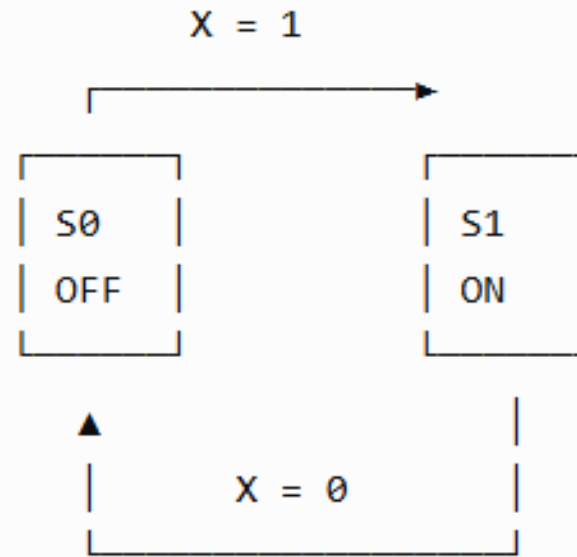
Data-Flow Graph (DFG):



Here, $\times$ represents AND operation and $+$ represents OR operation.

Question: A digital system has two states: $S0 = \text{OFF}$ and $S1 = \text{ON}$.

If $X = 1$, the system becomes ON. If $X = 0$, the system becomes OFF. Draw the State Diagram.



Transitions:

- $S0 \rightarrow S1$ when $X = 1$
- $S1 \rightarrow S0$ when $X = 0$
- $S0 \rightarrow S0$ when $X = 0$
- $S1 \rightarrow S1$ when $X = 1$

Thus, the state of the system depends on the input X .

Question: For the Boolean function

$$Y = (A + B) \cdot C$$

Draw the Sequencing/Data-Flow Graph and explain one method of Behavioral Optimization.

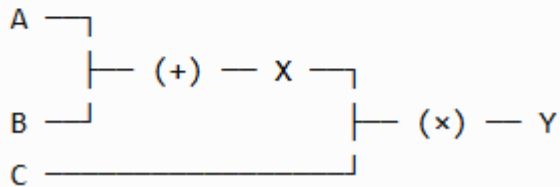
First, calculate:

$$X = A + B$$

Then calculate:

$$Y = X \cdot C$$

Data-Flow Graph:



Sequencing:

Step 1: $X = A + B$

Step 2: $Y = X \cdot C$

Behavioral Optimization:

Here, the two operations **cannot be performed in parallel**, because the second operation depends on the result of the first operation.

Therefore, an optimization method can be **operation simplification or efficient hardware/resource utilization** to reduce implementation cost or execution time.

Important: যদি question-এ behavioral optimization-এর **parallel operation** দেখাতে চান, তাহলে এমন function নেওয়া ভালো:

$$Y = (A + B) \cdot (C + D)$$

এখানে $A+B$ এবং $C+D$ independent, তাই দুটো operation parallel করা সম্ভব।

Q1. Hardware Modeling and HDL

What is Hardware Modeling? What is a Hardware Description Language (HDL)? Mention two advantages of using HDL in digital hardware design.

Q2. Logic Network and Data-Flow Graph

Consider the following Boolean function:

$$Y = [(A \cdot B) + (C \cdot D)] \cdot (E + F)$$

- (a) Draw the Logic Network.
- (b) Draw the Data-Flow Graph (DFG).

Q4. Sequencing, Data-Flow Graph and Behavioral Optimization

Consider the Boolean expression:

$$Y = [(A + B) \cdot (C + D)] + [(E \cdot F) + (G \cdot H)]$$

- (a) Divide the expression into elementary operations and draw the Data-Flow/Sequencing Graph.
- (b) Identify the operations that can be performed independently in parallel.
- (c) Explain how Behavioral Optimization can reduce the execution time.

Q3. State Diagram

A digital controller has three states:

- S_0 = IDLE
- S_1 = PROCESSING
- S_2 = COMPLETED

The controller follows these conditions:

- From S_0 , if Start = 1, it goes to S_1 .
- From S_1 , if Done = 1, it goes to S_2 .
- From S_1 , if Done = 0, it remains in S_1 .
- From S_2 , if Reset = 1, it returns to S_0 .
- Otherwise, it remains in S_2 .

Draw the complete State Diagram.